

LangGraph gRPC 架构详解

面向小白的完整指南

Python → gRPC → Go 调用链路与图执行位置全解析

2026-06-15

目录

1. 写给小白：什么是 gRPC?
2. 官方架构全景图
3. Python 端：gRPC 客户端
4. Go 端：核心服务
5. 完整调用链路（带时序图）
6. 核心问题：图在哪里执行？
7. Checkpoint 存储机制
8. 流事件分发机制
9. 总结与对比

1. 写给小白：什么是 gRPC?

在深入 LangGraph 架构之前，我们需要先理解 gRPC 是什么。本节假设你完全没有 gRPC 背景。

1.1 传统方式：REST API

你可能熟悉 REST API：客户端发送 HTTP 请求，服务器返回 JSON 响应。

```
# 客户端
response = requests.get('https://api.example.com/threads/123')
thread = response.json() # 解析 JSON

# 服务端
@app.get('/threads/{thread_id}')
def get_thread(thread_id):
    return {'id': thread_id, 'name': 'My Thread'}
```

1.2 gRPC 方式：像调用本地函数一样

gRPC 让你像调用本地函数一样调用远程服务：

```
# 客户端 - 看起来像本地函数调用!
thread = await client.threads.Get(thread_id='123')
# 返回的是强类型对象，不是 JSON 字典

# 服务端 - 实现一个接口
class ThreadsService:
    async def Get(self, request, context):
        return Thread(id='123', name='My Thread')
```

1.3 gRPC 的核心概念

概念	说明	类比
Protocol Buffers	定义消息格式的语言，比 JSON 更小更快	像提前定义好的表格模板
Service / Method	定义可调用的远程方法	像 REST 的 API 端点
Stub / Client	客户端生成的调用代理	像电话的拨号键盘
Channel	客户端到服务器的连接	像电话线
Request/Response	请求和响应消息	像电话通话内容

1.4 为什么要用 gRPC?

- 性能：二进制传输，比 JSON 快 5-10 倍
- 类型安全：编译时检查，不会拼错字段名
- 双向流：支持服务端推送，适合实时场景
- 跨语言：Python 调 Go 服务，像调本地函数

1.5 LangGraph 中的应用

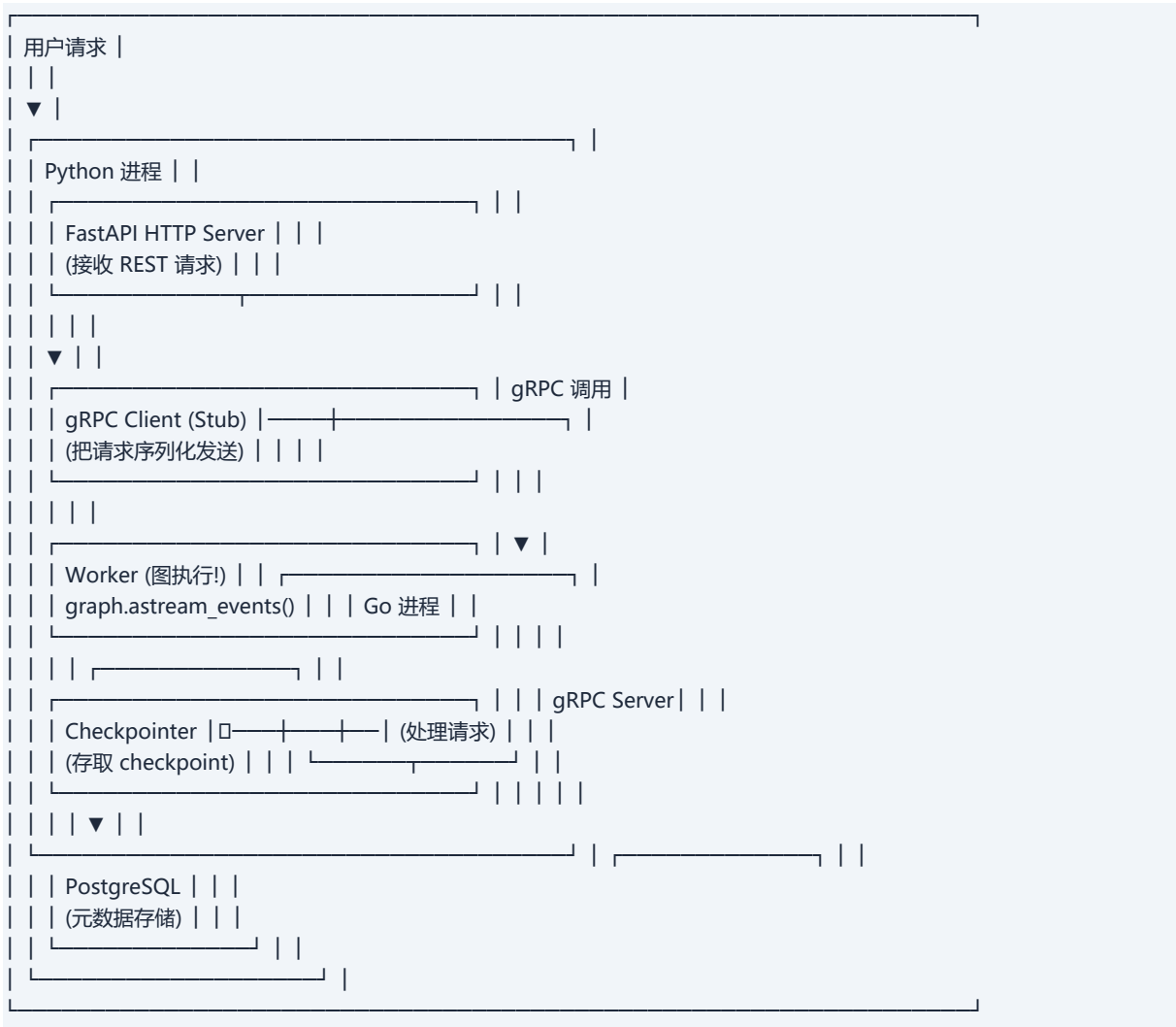
官方 LangGraph 使用 gRPC 让 Python API 层 调用 Go 持久化服务：

```
# Python 客户端调用 Go 服务
client = await get_shared_client() # 获取 gRPC 客户端
thread = await client.threads.Get( # 调用 Go 的 Get 方法
pb.GetThreadRequest(thread_id='123')
)
# 看起来像本地调用，实际通过网络发到了 Go 服务
```

2. 官方架构全景图

LangGraph Postgres Runtime 采用 Python + Go 双进程架构：

2.1 架构图



2.2 两个进程的职责

进程	语言	职责	开源状态
Python 进程	Python	HTTP API、图执行、Checkpoint 存取	□ 完全开源
Go 进程	Go	元数据 CRUD、流事件分发、TTL 清理	□ 闭源二进制

关键发现：图的实际执行在 Python 进程中，Go 进程只负责元数据管理和事件分发！

3. Python 端：gRPC 客户端

3.1 客户端连接池 (client.py)

Python 端维护一个 gRPC 客户端池，通过轮询方式分发请求：

```
class GrpcClient:
    def __init__(self, server_address):
        # 创建到 Go 服务的连接
        self._channel = aio.insecure_channel(server_address)

        # 创建各种服务的 Stub (代理)
        self._threads_stub = ThreadsStub(self._channel)
        self._runs_stub = RunsStub(self._channel)
        self._crons_stub = CronsStub(self._channel)
        # ... 更多 stub

class GrpcClientPool:
    """多个客户端实例，负载均衡"""
    def __init__(self, pool_size=5):
        self.clients = [GrpcClient() for _ in range(pool_size)]

    async def get_client(self):
        # 轮询选择客户端
        return self.clients[self._current_index % self.pool_size]
```

3.2 调用示例：获取 Thread

当 API 收到 GET /threads/{thread_id} 请求时：

```
# 第一步：API 路由层 (api/threads.py)
@router.get('/threads/{thread_id}')
async def get_thread(thread_id: str):
    async with connect() as conn:
        thread = await Threads.get(conn, thread_id) # 调用 ops
    return thread

# 第二步：gRPC ops 层 (grpc/ops/threads.py)
class Threads:
    @staticmethod
    async def get(conn, thread_id, ctx=None):
        # 构造 protobuf 请求
        request = pb.GetThreadRequest(
            thread_id=pb.UUID(value=str(thread_id))
        )

        # 发送 gRPC 请求到 Go 服务
        client = await get_shared_client()
        response = await client.threads.Get(request)

        # 转换响应为 Python dict
        yield proto_to_thread(response.thread)
```

3.3 gRPC vs 直连数据库

操作类型	Python 处理方式	Go 处理方式
Thread 元数据 CRUD	通过 gRPC 发请求	执行 SQL 操作 PostgreSQL
Run 元数据 CRUD	通过 gRPC 发请求	执行 SQL 操作 PostgreSQL
Checkpoint 读写	直接调用 Checkpointer	不参与 (或通过反向 gRPC 调 Python)
图执行	直接执行 graph.astream()	不参与
流事件分发	通过 gRPC 发送事件	分发给 SSE/WebSocket 订阅者

4. Go 端：核心服务

Go 服务是官方闭源的二进制文件，我们从 Python 端的调用推断其功能。

4.1 Go 服务提供的 gRPC 接口

Service	主要方法	功能
Threads	Get, Search, Create, Patch, Delete	线程元数据管理
Runs	Get, Search, Create, Delete, Next, Enter	Run 管理 + 队列
Assistants	Get, Search, Create, Patch, Delete	助手管理
Crons	Get, Search, Create, Delete, Next	定时任务管理
Cache	Get, Set, Delete	缓存管理
Admin	Migrate, Health	运维管理

4.2 Go 服务的数据库操作

Go 服务直接操作 PostgreSQL，处理元数据：

```
-- Go 服务执行的典型 SQL (推断)
SELECT * FROM threads WHERE thread_id = $1;
INSERT INTO runs (run_id, thread_id, status, ...) VALUES (...);
UPDATE threads SET status = 'busy', updated_at = NOW() WHERE thread_id = $1;
```

4.3 Go 服务不做什么

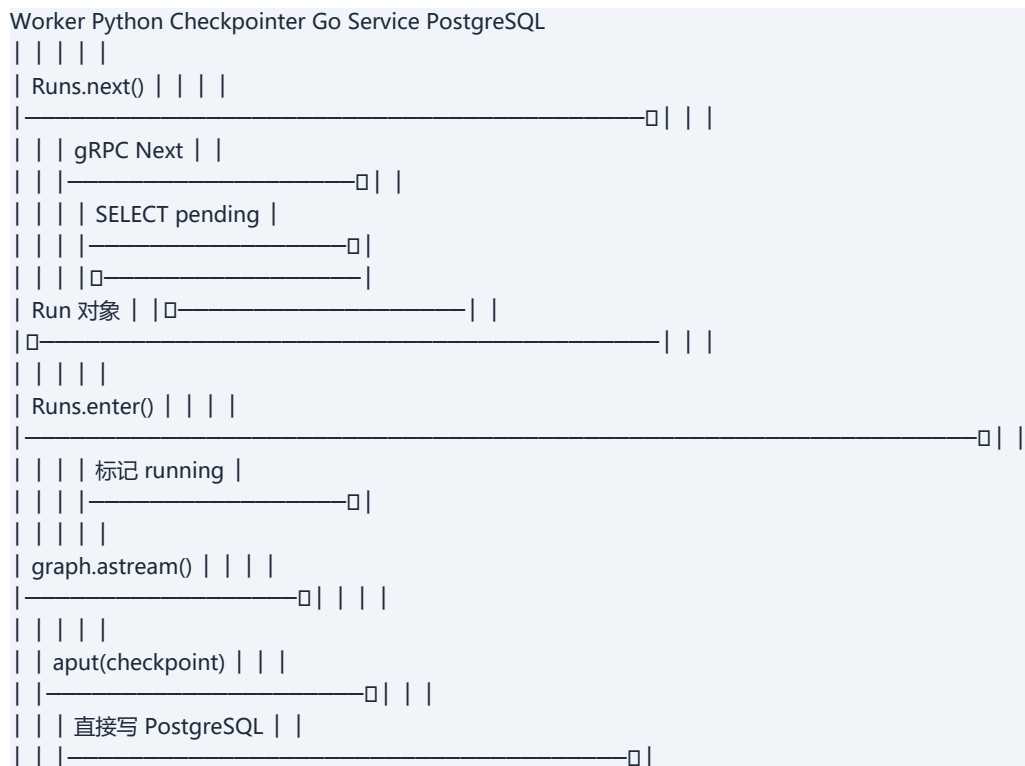
- 不执行图：图执行在 Python Worker 中
- 不直接操作 Checkpoint：通过反向 gRPC 调 Python
- 不解析用户代码：用户图定义在 Python 端

5. 完整调用链路（带时序图）

5.1 创建 Run 的完整流程



5.2 执行 Run 的完整流程



| | | |

| Runs.Stream.publish(event) | | |

| _____ □ | |

| | | | 分发到订阅者 |

| | | |

6. 核心问题：图在哪里执行？

答案：图在 Python 进程的 Worker 中执行，Go 服务完全不参与图的执行！

6.1 图执行的代码证据

在 worker.py 中，我们找到了图执行的核心代码：

```
# worker.py (Python 端)
async def worker(run: Run, attempt: int, ...):
# 1. 获取图定义 (Python 对象)
async with get_graph(graph_id, config, checkpoint=checkpoint) as graph:

# 2. 执行图！ (这是纯 Python 操作)
async for event in graph.astream_events(input, config, ...):
# 3. 处理每个事件
if event['event'] == 'on_chain_start':
# 节点开始执行
elif event['event'] == 'on_chain_end':
# 节点执行完成

# 4. 发布事件给订阅者
await Runs.Stream.publish(run_id, event_type, payload)
```

6.2 为什么图必须在 Python 执行？

- 用户代码在 Python：图的节点是 Python 函数，Go 无法执行
- LangChain 在 Python：LLM 调用、工具调用都是 Python 库
- 状态类型在 Python：TypedDict、Pydantic 模型都是 Python 类型

6.3 Go 服务的真正角色

Go 服务是一个元数据管理和事件分发中间件，不是执行引擎：

Go 服务角色	具体功能
元数据存储	管理 Thread、Run、Assistant、Cron 的元数据（存在 PostgreSQL）
任务队列	维护 pending runs 队列，Worker 通过 Runs.next() 拉取
事件分发	接收 Python 发布的事件，分发给 SSE/WebSocket 订阅者
TTL 清理	后台清理过期的 Thread 和 Run
健康检查	监控服务状态

7. Checkpoint 存储机制

Checkpoint 是图执行过程中的状态快照，用于暂停/恢复、回滚等功能。

7.1 Checkpoint 存储位置

- Checkpoint 数据存储在 PostgreSQL 的 checkpoint 相关表中
- 表由 langgraph-checkpoint-postgres 包管理 (Python 库)
- Go 服务 不直接操作 checkpoint 表

7.2 Python 写入 Checkpoint

图执行时，Python Checkpointer 直接写入 PostgreSQL：

```
# 图执行过程中，自动触发 checkpoint 保存
async def on_checkpoint(checkpoint):
    checkpointer = await get_checkpointer()
    await checkpointer.aput(config, checkpoint, metadata)
# 直接写入 PostgreSQL，不经过 Go 服务
```

7.3 Go 读取 Checkpoint (反向 gRPC)

当 Go 服务需要读取 checkpoint (如恢复执行) 时，会反向调用 Python：

```
# Python 端暴露的 gRPC 服务 (grpc/servicers/checkpointer.py)
class CheckpointerServicerImpl:
    async def GetTuple(self, request, context):
        # Go 服务调用这个方法来获取 checkpoint
        checkpointer = await get_checkpointer()
        result = await checkpointer.aget_tuple(config)
        return GetTupleResponse(checkpoint_tuple=result)

    async def Put(self, request, context):
        # Go 服务调用这个方法保存 checkpoint
        await checkpointer.aput(config, checkpoint, metadata)
```

7.4 双向 gRPC 通信

Python → Go (正向):

```
client.threads.Get() # 获取线程元数据
client.runs.Next() # 获取下一个待执行 run
client.runs.Publish() # 发布流事件
```

Go → Python (反向):

```
CheckpointerServicer.GetTuple() # Go 读取 checkpoint
CheckpointerServicer.Put() # Go 写入 checkpoint
```

8. 流事件分发机制

当图执行产生事件时，需要实时推送给订阅者（如 LangGraph Studio）。

8.1 事件产生与发布

```
# Python Worker 执行图
async for event in graph.astream_events(input, config):
    # event = {'event': 'on_chain_start', 'name': 'agent', 'data': {...}}

    # 序列化事件
    payload = json_dumps(event)

    # 通过 gRPC 发布到 Go 服务
    await Runs.Stream.publish(
        run_id=run_id,
        event='values', # 事件类型
        message=payload,
        thread_id=thread_id,
    )
```

8.2 Go 服务分发事件

Go 服务接收事件后，分发给所有订阅者：

```
# Go 服务内部逻辑（推断）
func (s *RunsServer) Publish(ctx context.Context, req *PublishRequest) {
    // 存储到 Redis Stream（可恢复）
    s.redis.XAdd(ctx, &redis.XAddArgs{
        Stream: fmt.Sprintf("run:%s:events", req.RunId),
        Values: map[string]interface{}{
            "event": req.Event,
            "message": req.Message,
        },
    })

    // 推送到 WebSocket/SSE 订阅者
    for _, sub := range s.subscribers[req.RunId] {
        sub.Send(req.Message)
    }
}
```

8.3 客户端订阅事件

```
# 客户端通过 SSE 订阅
GET /threads/{thread_id}/runs/{run_id}/stream

# 响应是 SSE 事件流
event: values
data: {"event": "on_chain_start", "name": "agent", ...}

event: values
data: {"event": "on_chain_end", "name": "agent", ...}
```

9. 总结与对比

9.1 核心结论

图在 Python 进程执行，Go 服务只负责元数据管理和事件分发。

9.2 Python vs Go 职责划分

职责	Python 进程	Go 进程
HTTP API	☐ 处理请求	☐ 不参与
图执行	☐ 执行 graph.astream()	☐ 不参与
Checkpoint 读写	☐ 直接操作 PostgreSQL	☐ 通过反向 gRPC 调 Python
Thread/Run 元数据	☐ 通过 gRPC 调 Go	☐ 直接操作 PostgreSQL
流事件分发	☐ 发送到 Go	☐ 分发给订阅者
任务队列	☐ 拉取任务	☐ 维护队列
TTL 清理	☐ 不参与	☐ 后台定时清理

9.3 为什么官方用 Go?

- 性能：Go 处理大量并发连接更高效
- 数据库连接池：Go 的 pgx 库性能优秀
- 部署独立：可以单独扩展持久化层
- 商业保护：核心持久化逻辑闭源

9.4 自研 postgres_py 的优势

- 完全开源：无需依赖闭源 Go 二进制
- 简单部署：单进程，无需 Docker
- 易于调试：纯 Python，IDE 断点无障碍
- 学习友好：理解每个细节如何工作